

Customer Churn Prediction ML Model via Random Forest Classifier

```
main_RandomForestClassifier.ipynb
main_RandomForestClassifier.py
saas_customer_data.csv

# ===== 1. Load and Inspect Data =====
import pandas as pd

df = pd.read_csv('saas_customer_data.csv')
print(df.head())
print(df.info())
print("Missing values:\n", df.isnull().sum())

# ===== 2. Data Preprocessing =====
from sklearn.preprocessing import StandardScaler

# Drop CustomerID (non-informative)
df.drop(columns=['CustomerID'], inplace=True)

# Encode Gender
df['Gender'] = df['Gender'].map({'Male': 0, 'Female': 1})

# One-hot encode SubscriptionType
df = pd.get_dummies(df, columns=['SubscriptionType'], drop_first=True)

# Separate features and target
X = df.drop('Churn', axis=1)
y = df['Churn']

# Scale numeric features (optional for tree models, but we do it for consistency)
scaler = StandardScaler()
X[['Age', 'MonthlyUsageHours', 'SupportTickets']] = scaler.fit_transform(
    X[['Age', 'MonthlyUsageHours', 'SupportTickets']]
)
```

```
# ===== 3. Train/Test Split =====
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)

# ===== 4. Model Training (Random Forest) =====
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, roc_curve

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)
y_proba = rf_model.predict_proba(X_test)[:, 1]

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("ROC AUC Score:", roc_auc_score(y_test, y_proba))
```

```

# ===== 5. Feature Importance Visualization =====
import matplotlib.pyplot as plt
import seaborn as sns

feature_importances = pd.Series(rf_model.feature_importances_, index=X.columns)
feature_importances_sorted = feature_importances.sort_values(ascending=False)

sns.barplot(x=feature_importances_sorted.values, y=feature_importances_sorted.index)
plt.title("Random Forest Feature Importances")
plt.xlabel("Importance Score")
plt.ylabel("Feature")
plt.show()

# ===== 6. ROC Curve Visualization =====
fpr, tpr, thresholds = roc_curve(y_test, y_proba)

plt.figure(figsize=(8, 5))
plt.plot(fpr, tpr, label=f'AUC = {roc_auc_score(y_test, y_proba):.2f}')
plt.plot([0, 1], [0, 1], '--', color='gray')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()

```

```

# ===== 7. Export Results for Stakeholders =====
results_df = X_test.copy()
results_df['ActualChurn'] = y_test
results_df['PredictedChurn'] = y_pred
results_df['ChurnProbability'] = y_proba

results_df.to_csv('churn_rf_predictions.csv', index=False)
print("Prediction results saved to 'churn_rf_predictions.csv'")

```

	CustomerID	Age	Gender	SubscriptionType	MonthlyUsageHours	SupportTickets	\
0	CUST0000	56	Female	Basic	28.9	2	
1	CUST0001	69	Female	Basic	36.4	1	
2	CUST0002	46	Male	Enterprise	61.0	2	
3	CUST0003	32	Female	Pro	43.3	1	
4	CUST0004	60	Female	Pro	37.5	2	

Churn

0	1
1	0
2	0
3	1
4	0

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1000 entries, 0 to 999

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	CustomerID	1000 non-null	object
1	Age	1000 non-null	int64
2	Gender	1000 non-null	object
3	SubscriptionType	1000 non-null	object
4	MonthlyUsageHours	1000 non-null	float64
5	SupportTickets	1000 non-null	int64
6	Churn	1000 non-null	int64

...

macro avg	0.53	0.52	0.52	300
weighted avg	0.66	0.67	0.66	300

ROC AUC Score: 0.615536010824774

